

1.5em 0pt

Plantilla de artículo IEEE en LaTeX: guía práctica para estudiantes

Karen C. Villaronga F.
Facultad de Ingeniería Mecatrónica
Universidad Santo Tomás
Bucaramanga, Colombia
karencecilia.villaronga@ustabuca.edu.co

Joseph S. Suarez B.
Facultad de Ingeniería Mecatrónica
Universidad Santo Tomás
Bucaramanga, Colombia
josephsuarez@ustabuca.edu.co

Abstract—This document is a template in Spanish for writing articles in LaTeX using the IEEE format. It briefly explains the objective of the work, the general methodology, and the main results obtained.

Index Terms—LaTeX, IEEEtran, templates, figures, tables, bibliography.

RESUMEN

Este documento es una plantilla en español para escribir artículos en LaTeX con el formato IEEE. Aquí se explica brevemente el objetivo del trabajo, la metodología general y el resultado principal.

PALABRAS CLAVE

LaTeX, IEEEtran, plantillas, figuras, tablas, bibliografía.

I. INTRODUCCIÓN

En los artículos originales enviados a la revista IEEE, la introducción debe presentar de forma concisa el contexto y la relevancia del problema abordado, describiendo brevemente el área de estudio y su importancia en el campo de la ingeniería o tecnología; incluir un resumen del estado del arte con referencia a los trabajos más relevantes y recientes, destacando las limitaciones o vacíos existentes que motivan la investigación; exponer con claridad el objetivo y la principal contribución del trabajo, indicando qué se propone y en qué se diferencia o mejora respecto a lo ya publicado; y, opcionalmente, cerrar con una breve descripción de la estructura del artículo para guiar al lector en el desarrollo del contenido.

La escritura académica en LaTeX ofrece control tipográfico y reproducibilidad. Esta plantilla en español muestra, con ejemplos mínimos, cómo estructurar un artículo usando IEEEtran: figuras, tablas, ecuaciones y bibliografía. Úsala para practicar el flujo básico de edición y compilación.

El resto del documento se organiza así: en la Sección ?? se describe un ejemplo de desarrollo experimental simplificado; en la Sección ?? se explica la metodología con ecuaciones de ejemplo; en la Sección VI se muestran tablas y figuras de ejemplo; y en la Sección VII se listan conclusiones y trabajo futuro.

LaTeX facilita gestionar referencias con BibTeX, por ejemplo [?], [?]. También puedes usar estilos numéricos compatibles con IEEE. Las citas deben registrarse en el archivo `reference.bib`

Diversos trabajos describen técnicas de análisis espectral y aprendizaje automático aplicadas a señales [?], [?]. Ajusta esta sección a tu tema específico.

II. MARCO REFERENCIAL

A. Actividad 1

Para esta actividad, se requiere plantear el diseño un sistema embebido basado en un microcontrolador que gestione cinco tareas con diferentes requisitos temporales: la lectura de un sensor de alta velocidad (cada 10 ms), el control de un motor derivado de dicha lectura, el envío de telemetría vía serial (cada 100 ms), la actualización de una pantalla informativa y la atención inmediata de un botón de emergencia. El reto técnico consiste en garantizar que las tareas lentas (como la pantalla) no interfieran con las críticas (sensor y emergencia).

En la Figura 1 se observa el diagrama del problema descrito, junto con la estimación del tiempo total de ejecución del ciclo secuencial. La suma de las tareas da aproximadamente 75 ms por ciclo completo. Esto implica que, aunque el sensor debe leerse cada 10 ms, el ciclo completo tarda 75 ms, por lo que se pierden 6 de cada 7 lecturas. Asimismo, el botón de emergencia puede tardar hasta 75 ms en ser atendido, lo cual resulta inaceptable en un entorno industrial.

El reto técnico consiste en garantizar que las tareas lentas (como la pantalla) no interfieran con las críticas (sensor y emergencia).

1) Discusión del caso

En un entorno secuencial, todo el control reside en un bucle infinito donde el procesador ejecuta una instrucción tras otra. Para gestionar los tiempos, se suelen usar banderas o

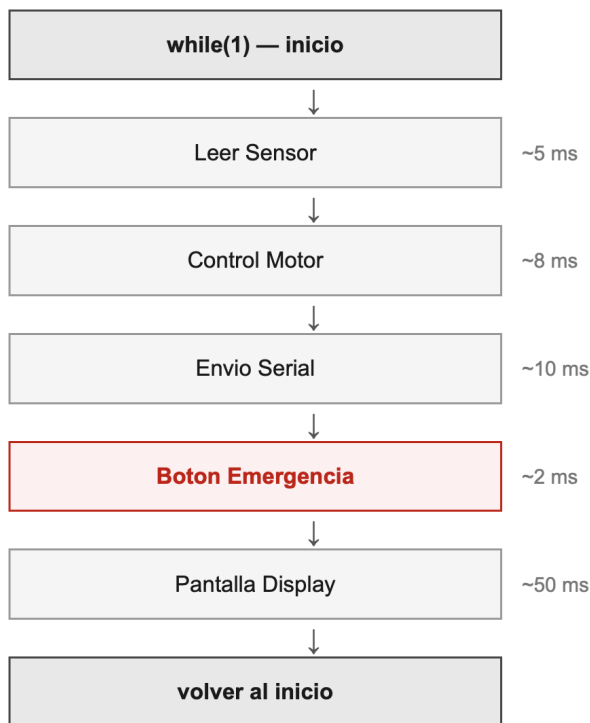


Figure 1. Diagrama del sistema secuencial con `while(1)` y tiempos acumulados por ciclo.

contadores de tiempo (como `millis()`). El código pregunta constantemente si es hora de leer el sensor o enviar datos, ejecutando las funciones solo cuando se cumple el tiempo establecido.

Sin embargo, el mayor riesgo de esto es el bloqueo por latencia. Si la tarea de "Actualizar pantalla" tarda 30 ms en completarse, el procesador quedará atrapado en esa función, impidiendo que el sensor se lea cada 10 ms. Esto genera una pérdida de datos y un control del motor errático. Además, el sistema se vuelve rígido, pues cualquier cambio en una tarea afecta el tiempo de todas las demás.

En esta tarea, la prioridad absoluta es el botón de emergencia, la cual es una medida de seguridad, por lo que no puede esperar a que un bucle termine su vuelta. Si el sistema está ocupado enviando un mensaje largo por el puerto serial y ocurre un accidente, un retraso de milisegundos en la detección del botón podría tener consecuencias catastróficas para el hardware o el operador [1].

Para asegurar el cumplimiento de los tiempos, es necesario transitar hacia una arquitectura orientada a eventos. Esto implica usar interrupciones para el botón de emergencia y los timers del sensor. Un RTOS facilitaría esto asignando

prioridades, permitiendo que las tareas urgentes "expulsen" a las tareas lentas (como la pantalla) del CPU cuando sea necesario [2] [1].

2) Solución propuesta

La solución propuesta consiste en migrar de la arquitectura Super-loop a un RTOS (como FreeRTOS), donde cada tarea se ejecuta de forma independiente con su propio periodo y nivel de prioridad asignado por un Scheduler [2] [1]. El botón de emergencia deja de ser una línea más dentro del bucle y se convierte en una interrupción hardware (ISR, del inglés, *"Interrupt Service Routine"* o Rutina de Servicio de Interrupción), garantizando una respuesta en microsegundos sin importar qué esté haciendo el sistema [2]. El sensor se ejecuta como tarea periódica estricta cada 10 ms, el control del motor reacciona inmediatamente tras cada lectura, el envío serial corre cada 100 ms de forma independiente, y el display opera como tarea de fondo con la menor prioridad, de modo que sus 50 ms de ejecución nunca bloquean las tareas críticas.

Así, el Scheduler se encarga de ceder y recuperar el CPU entre tareas según su urgencia, eliminando el problema central del Super-loop: que el tiempo de respuesta de cualquier evento dependa del tiempo de ciclo total del sistema.

3) Respuesta Central: ¿Por qué no es suficiente un `while(1)`?

Programar todo en un `while(1)` (conocido como arquitectura Super-loop) no es suficiente para sistemas complejos por tres razones críticas [2].

En primer lugar, existe una falta de determinismo: el tiempo que tarda una vuelta completa del bucle depende de qué condiciones `if` se ejecutaron. Si la escritura en pantalla tarda 50 ms, la lectura del sensor, que debe realizarse cada 10 ms, se retrasará inevitablemente.

En segundo lugar, se produce una latencia inaceptable. Los eventos urgentes deben esperar a que el puntero del programa llegue a la línea donde se evalúa la condición correspondiente. En entornos de seguridad industrial, un retardo de 100 ms puede ser suficiente para provocar un accidente.

Por último, se puede destacar que la arquitectura Super-loop es inescalable. A medida que se agregan funcionalidades, el código se vuelve frágil, por lo que modificar un simple `delay()` en una función puede comprometer el tiempo de respuesta de todo el sistema [2].

En la Figura 2 se presenta el diagrama de la solución prop-

uesta basada en un esquema con prioridades e interrupciones. El botón de emergencia se atiende mediante una interrupción hardware de máxima prioridad ($ISR < 1 \mu s$), garantizando respuesta inmediata. La lectura del sensor se ejecuta como tarea periódica cada 10 ms, seguida del control del motor como tarea reactiva. El envío serial se programa cada 100 ms, mientras que la actualización de la pantalla se ejecuta como tarea de fondo cuando el procesador está libre. Este enfoque asegura que las tareas críticas no sean bloqueadas por las de menor prioridad.

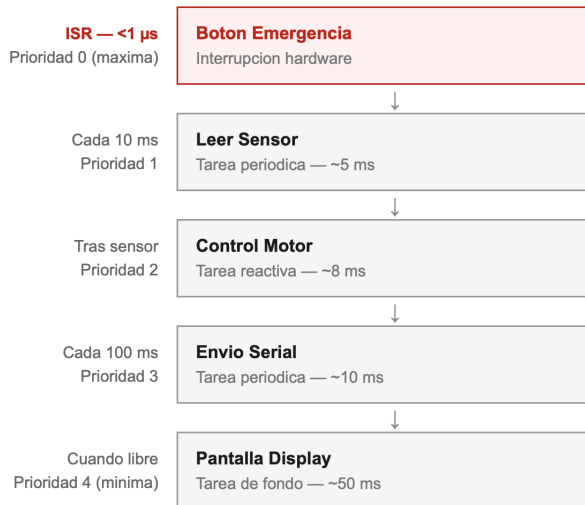


Figure 2. Arquitectura de solución basada en prioridades e interrupciones.

4) Material de consulta

a) Interrupciones

- ¿Qué son?: Son mecanismos de hardware que permite al procesador suspender temporalmente la ejecución del programa principal para atender un evento urgente y específico [3].
- ¿Cómo funcionan en un microcontrolador?: Actúa cuando ocurre un evento, el hardware completa la instrucción actual, guarda el estado del CPU en la pila (stack) y salta a una dirección de memoria fija llamada Vector de Interrupción, donde ejecuta la ISR (Rutina de Servicio de Interrupción). Al terminar, restaura el estado y vuelve al punto exacto donde se quedó [3].
- ¿Qué problemas resuelven?: El Polling (preguntar constantemente). Sin interrupciones, el CPU pierde tiempo preguntando "¿ya presionaron el botón?" miles de veces por segundo [3].
- ¿Qué riesgos implican?: Pueden causar corrupción de

datos si no se manejan variables atómicas y, si son muy frecuentes, pueden provocar que el programa principal nunca se ejecute [3].

b) Planificación y Scheduler

- ¿Qué es un scheduler?: Es el componente del núcleo (Kernel) encargado de decidir qué tarea (hilo) tiene el control del CPU en un momento dado [4].
- ¿Qué significa prioridad?: Es un valor numérico asignado a cada tarea. El Scheduler siempre favorecerá la ejecución de la tarea con mayor prioridad [4].
- ¿Qué es planificación preventiva y cooperativa?:
 - Preventiva: El Scheduler puede "quitarle" el CPU a una tarea a la fuerza si aparece una de mayor prioridad. Es ideal para tiempo real.
 - Cooperativa: Una tarea debe terminar o "ceder" voluntariamente el CPU para que otra pueda entrar. Si una tarea se queda en un bucle infinito, bloquea todo el sistema [4].

c) Multitarea

- ¿Qué es concurrencia?: Es la capacidad del sistema para manejar múltiples tareas que parecen progresar simultáneamente, aunque en un solo núcleo se ejecuten una por una intercaladas [3] [5].
- Diferencia entre multitarea real y simulada:
 - Real: Ocurre en procesadores de varios núcleos (Multi-core), donde cada núcleo ejecuta una tarea distinta físicamente al mismo tiempo.
 - Simulada: Ocurre en microcontroladores de un solo núcleo mediante el Cambio de Contexto (Context Switching), alternando tareas tan rápido que el ojo humano no percibe la pausa [5] [1].
- Ejemplos en sistemas embebidos:
 - Un sistema con RTOS como FreeRTOS donde:
 - * Una tarea maneja WiFi.
 - * Otra controla motores.
 - * Otra procesa datos de sensores [1].
 - En un ESP32 (doble núcleo):
 - * Núcleo 0 ejecuta la pila WiFi.

- * Núcleo 1 ejecuta la lógica principal del usuario [5].

d) Sincronización

- ¿Qué es una condición de carrera?: Ocurre cuando dos o más tareas intentan modificar un recurso compartido (como una variable global) al mismo tiempo, dejando un resultado impredecible [6].
- ¿Qué son semáforos y mutex?:
 - Mutex: Abreviatura del inglés "*mutual exclusion*" o exclusión mutua, es un "candado". Solo una tarea puede tener la llave para entrar a la sección crítica [6].
 - Semáforo: Es un "contador". Permite que un número limitado de tareas accedan a un recurso o sirve para señalar entre tarea [6].
- ¿Qué es sección crítica?: Es la parte del código que accede a ese recurso compartido y que no debe ser interrumpida por ninguna otra tarea que use el mismo recurso [3] [6].

III. DESARROLLO DE TALLERES

A. Primer taller: Programación estructurada en sistemas embebidos

Este taller integrador se centra en el diseño y desarrollo de sistemas embebidos bajo el enfoque de la programación estructurada, integrando el modelado formal del sistema con su implementación práctica. A través de tres retos aplicados, el objetivo principal es desarrollar la capacidad de estructurar soluciones embebidas desde una perspectiva ingenieril, donde se debe justificar la selección del microcontrolador, organizar el código de forma modular y documentada, e implementar buenas prácticas en todo el desarrollo del mismo.

1) RETO 1 – Sistema de Control de Acceso

- Análisis del problema:
El sistema de control de acceso tiene como propósito verificar una contraseña digital ingresada mediante una combinación de botones físicos o un teclado matricial. Si la combinación es correcta, se activa una cerradura electrónica (servo motor o relé); en caso contrario, se contabilizan los intentos fallidos y, al superar el límite permitido, el sistema entra en un estado de bloqueo temporal.
- Identificación de Entradas y Salidas

- * Entradas del sistema:

- Botones digitales (4 pulsadores) o teclado matricial 4x4 para ingreso de contraseña.
- Señal de reset (opcional): botón para reiniciar el sistema desde estado bloqueado.

– Restricciones Técnicas

- * Máximo 3 intentos fallidos antes de activar el bloqueo.
- * Tiempo de bloqueo: 30 segundos.
- * La contraseña debe almacenarse en memoria no volátil (EEPROM) o en código.
- * Temporización no bloqueante mediante millis()
- * El sistema debe retornar automáticamente al estado de espera tras el desbloqueo.

– Variables Involucradas

Table I
VARIABLES DEL SISTEMA DE CONTROL DE ACCESO

Variable	Tipo	Descripción
intentosFallidos	int	Contador de intentos incorrectos (0-3)
contrasenaIngresada	String	Secuencia de teclas presionadas por el usuario
contrasenaCorrecta	String	Contraseña almacenada en firmware
sistemaBloqueado	bool	Indica si el sistema está en estado de bloqueo
tiempoBloqueo	unsigned long	Marca de tiempo de inicio del bloqueo
accesoGarantizado	bool	Indicador de acceso concedido

• Flujograma del sistema

A continuación se describe el algoritmo de comportamiento del sistema de control de acceso. El diagrama representa el flujo lógico completo desde el inicio hasta la resolución de cada evento de acceso.

– Descripción del Algoritmo (Pseudocódigo Estructurado):

En el Algoritmo 1 se puede observar el pseudocódigo, el cual, comienza declarando un bloque de INICIO que configura los periféricos. A continuación se evalúa si el sistema está bloqueado; en caso afirmativo, se verifica el temporizador y se libera al cumplirse 30 segundos. Si el sistema no está bloqueado, se espera la entrada del

usuario. Cuando la contraseña está completa, se ejecuta la comparación: si es correcta, se activa el servo y el LED verde; si es incorrecta, se incrementa el contador de fallos y se evalúa si se alcanzó el límite. El proceso retorna al inicio del bucle en todos los casos.

Algorithm 1 Algoritmo de Control de Acceso

```

Inicialización:
1: Inicializar pines, variables y servo en posición cerrada
2: intentosFallidos = 0
3: sistemaBloqueado = false
4: while true do
5:   if sistemaBloqueado then
6:     if millis() - tiempoBloqueo ≥ 30000 then
7:       sistemaBloqueado = false
8:       intentosFallidos = 0
9:       Apagar LED_ROJO
10:    end if
11:  else
12:    Leer tecla del teclado
13:    Agregar caracter a contrasenaIngresada
14:    if |contrasenaIngresada| = |contrasenaCorrecta| then
15:      if contrasenaIngresada = contrasenaCorrecta then
16:        Activar servo (abrir cerradura)
17:        Encender LED_VERDE durante 3 segundos
18:        Regresar servo a posición cerrada
19:        intentosFallidos = 0
20:      else
21:        intentosFallidos = intentosFallidos + 1
22:        Hacer parpadear LED_ROJO
23:        if intentosFallidos ≥ 3 then
24:          sistemaBloqueado = true
25:          tiempoBloqueo = millis()
26:        end if
27:      end if
28:      contrasenaIngresada = ""
29:    end if
30:  end if
31: end while

```

• Justificación técnica del microcontrolador seleccionado

- Microcontrolador Seleccionado: Arduino UNO (ATmega328P): El Arduino UNO con microcontrolador ATmega328P resulta adecuado para este sistema debido a su disponibilidad, bajo costo, amplia comunidad de soporte y suficiente cantidad de pines para los periféricos requeridos[7].

- Análisis de Pines Requeridos: El ATmega328P dispone de 14 pines digitales (6 con PWM) y 6 analógicos, cubriendo ampliamente los 12 pines requeridos con margen de expansión.

Table II
ANÁLISIS DE PINES REQUERIDOS

Componente		Pines Requeridos	Tipo
Teclado	matricial 4x4	8 pines digitales (4 filas + 4 columnas)	Digital I/O
LED verde		1 pin digital	Digital OUTPUT
LED rojo		1 pin digital	Digital OUTPUT
Servo motor		1 pin PWM	PWM OUTPUT
Buzzer		1 pin digital	Digital OUTPUT
Total		12 pines	—

– Capacidad de Procesamiento

- * CPU: ATmega328P @ 16 MHz
- * Memoria Flash: 32 KB (programa ocupa < 5 KB)
- * SRAM: 2 KB (suficiente para buffers de contraseña y variables de estado)
- * EEPROM: 1 KB (almacenamiento persistente de contraseña)

– Consumo Energético

- * Consumo activo: 46 mA @ 5V = 230 mW
- * LED: 20 mA c/u con resistencia limitadora de 220 ohms
- * Servo SG90: 100–250 mA en movimiento
- * Alimentación recomendada: 5V/1A (USB o adaptador)

– Componentes Auxiliares

– Esquema de Conexión

En la Figura 3 se observa el diagrama de conexión correspondiente al funcionamiento de este circuito, desarrollado a través del programa de circuitado. A continuación se mencionan las conexiones establecidas en el proyecto:

- * Teclado 4x4: Filas a pines D4, D5, D6, D7 — Columnas a pines D8, D9, D10, D11
- * LED verde: Pin D12 → Resistencia 220 ohms →

Table III
COMPONENTES ELECTRÓNICOS DEL SISTEMA

Componente	Cant.	Especificación / Función
LED verde 5mm	1	3.3V, 20mA. Indicador de acceso correcto.
LED rojo 5mm	1	3.3V, 20mA. Indicador de error o bloqueo.
Resistencia 220 Ω	2	1/4 W. Limitación de corriente para los LEDs.
Buzzer pasivo	1	5V. Generación de señal sonora de retroalimentación.
Protoboard / PCB	1	830 puntos. Montaje físico del circuito.

Ánodo LED $\rightarrow$ Cátodo $\rightarrow$ GND

* LED rojo: Pin D13 $\rightarrow$ Resistencia 220 ohms $\rightarrow$ Ánodo LED $\rightarrow$ Cátodo $\rightarrow$ GND

* Servo SG90: Cable señal a Pin D3 (PWM), VCC a 5V, GND a GND

* Buzzer: Pin D2 $\rightarrow$ Buzzer (+) $\rightarrow$ GND

* Alimentación: 5V desde USB o regulador 7805

EasyEDA.

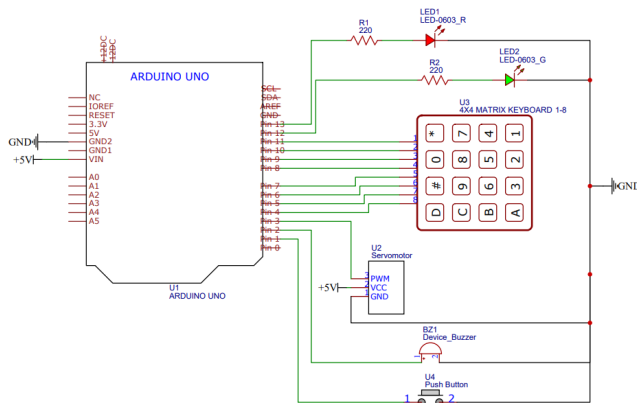


Figure 3. Esquémático de Conexión Reto 1

• Código documentado

El código está organizado de forma modular separando la inicialización de hardware, la lógica de verificación, el control de la cerradura y la gestión de estados de bloqueo. Se evita el uso de delay() para garantizar temporización no bloqueante.

Listing 1. Implementación de Control de Acceso con Teclado y Servo

```
#include <Keypad.h>
```

```
#include <Servo.h>
```

```
// --- Definicion de pines ---
```

```
#define PIN_SERVO      3
#define PIN_BUZZER     2
#define PIN_LED_VERDE  12
#define PIN_LED_ROJO   13
#define PIN_BOTON      A0
```

```
// --- Configuracion del teclado 4x4 ---
```

```
const byte FILAS      = 4;
const byte COLUMNAS   = 4;
```

```
char teclas[FILAS][COLUMNAS] = {
    { '1', '2', '3', 'A' },
    { '4', '5', '6', 'B' },
    { '7', '8', '9', 'C' },
    { '*', '0', '#', 'D' }
};
```

```
byte pinesFilas[FILAS] =
    { 11, 10, 9, 8 };
byte pinesColumnas[COLUMNAS] =
    { 7, 6, 5, 4 };
```

```
Keypad teclado =
    Keypad(makeKeymap( teclas ),
            pinesFilas ,
            pinesColumnas ,
            FILAS,
            COLUMNAS);
```

```
// --- Servo ---
```

```
Servo servo;
#define ANGULO_ABIERTO  90
#define ANGULO_CERRADO  0
```

```
// --- Variables de control de
acceso ---
```

```
const String PASSWORD = "1234";
const byte MAX_INTENTOS = 3;
const unsigned long TIEMPO_BLOQUEO
= 30000;
```

```
// --- Estado general ---
String entradaActual = "";
```

```

byte intentosFallidos = 0;
bool bloqueado = false;
unsigned long tiempoBloqueo = 0;

// --- Servo temporizado ---
bool puertaAbierta = false;
unsigned long tiempoServo = 0;

// --- Buzzer no bloqueante ---
bool buzzerActivo = false;
unsigned long tiempoBeep = 0;
int duracionBeep = 0;

// --- Parpadeo LED rojo ---
unsigned long tiempoBlink = 0;
bool estadoBlink = false;
byte parpadeosPendientes = 0;
bool ledRojoFijo = false;

// --- Anti-rebote boton ---
unsigned long ultimoDebounce = 0;
const unsigned long
DEBOUNCE_DELAY = 50;

// =====
void inicializarHardware() {
    pinMode(PIN_LED_VERDE, OUTPUT);
    pinMode(PIN_LED_ROJO, OUTPUT);
    pinMode(PIN_BUZZER, OUTPUT);
    pinMode(PIN_BOTON,
INPUT_PULLUP);
    servo.attach(PIN_SERVO);
    servo.write(ANGULO_CERRADO);
    digitalWrite(PIN_LED_VERDE, LOW);
    digitalWrite(PIN_LED_ROJO, LOW);
    Serial.println(" Sistema de
acceso listo");
}

// =====
void beep(int frecuencia,
          int duracion) {
    tone(PIN_BUZZER, frecuencia);
    buzzerActivo = true;
    tiempoBeep = millis();
    duracionBeep = duracion;

```

```

}

// =====
void actualizarBuzzer() {
    if (buzzerActivo) {
        if (millis() - tiempoBeep
            >= duracionBeep) {
            noTone(PIN_BUZZER);
            buzzerActivo = false;
        }
    }
}

// =====
void actualizarServo() {
    if (puertaAbierta) {
        if (millis() - tiempoServo
            >= 3000) {
            servo.write(ANGULO_CERRADO);
            digitalWrite(PIN_LED_VERDE,
LOW);
            puertaAbierta = false;
        }
    }
}

// =====
void actualizarBlink() {
    if (!estadoBlink) return;
    if (millis() - tiempoBlink < 300)
        return;
    tiempoBlink = millis();
    bool ledActual =
        digitalRead(PIN_LED_ROJO);
    if (!ledActual) {
        digitalWrite(PIN_LED_ROJO, HIGH);
        parpadeosPendientes--;
    } else {
        if (parpadeosPendientes == 0) {
            estadoBlink = false;
            digitalWrite(PIN_LED_ROJO,
                ledRojoFijo ? HIGH : LOW);
        } else {
            digitalWrite(PIN_LED_ROJO, LOW);
        }
    }
}

```



```

}

// =====
void accesoAprobado() {
    intentosFallidos = 0;
    estadoBlink       = false;
    ledRojoFijo       = false;
    digitalWrite(PIN_LED_ROJO, LOW);
    digitalWrite(PIN_LED_VERDE, HIGH);
    servo.write(ANGULO_ABIERTO);
    puertaAbierta = true;
    tiempoServo   = millis();
    beep(2000, 150);
}

// =====
void accesoNegado() {
    intentosFallidos++;
    if (intentosFallidos <
    MAX_INTENTOS)
    {
        parpadeosPendientes = 2;
        ledRojoFijo         = false;
    } else {
        parpadeosPendientes = 3;
        ledRojoFijo         = true;
    }
    estadoBlink = true;
    tiempoBlink = millis();
    digitalWrite(PIN_LED_ROJO, LOW);
    beep(500, 500);
    if (intentosFallidos >=
    MAX_INTENTOS)
    {
        bloqueado = true;
        tiempoBloqueo = millis();
    }
}

// =====
void verificarPassword() {
    if (entradaActual == PASSWORD)
        accesoAprobado();
    else
        accesoNegado();
    entradaActual = "";
}

```

```

}

// =====
void procesarTecla(char tecla) {
    if (tecla == '#') {
        verificarPassword();
        return;
    }
    if (tecla == '*') {
        entradaActual = "";
        beep(1000, 100);
        return;
    }
    entradaActual += tecla;
    if (entradaActual.length()
        > PASSWORD.length()) {
        entradaActual = "";
        beep(500, 200);
    }
}

// =====
void leerBoton() {
    if (digitalRead(PIN_BOTON) == LOW) {
        if (millis() - ultimoDebounce
            > DEBOUNCE_DELAY) {
            bloqueado = false;
            intentosFallidos = 0;
            entradaActual = "";
            estadoBlink = false;
            ledRojoFijo = false;
            digitalWrite(PIN_LED_ROJO, LOW);
            digitalWrite(PIN_LED_VERDE,
            LOW);
            beep(1800, 200);
            ultimoDebounce = millis();
        }
    }
}

// =====
void setup() {
    Serial.begin(9600);
    inicializarHardware();
}

```

```
// =====
void loop() {
    actualizarBuzzer();
    actualizarServo();
    actualizarBlink();
    leerBoton();
    if (bloqueado) {
        if (millis() - tiempoBloqueo
            >= TIEMPO_BLOQUEO) {
            bloqueado = false;
            intentosFallidos = 0;
            estadoBlink = false;
            ledRojoFijo = false;
            digitalWrite(PIN_LED_ROJO, LOW);
            beep(1500, 300);
        }
        return;
    }
    char tecla = teclado.getKey();
    if (tecla) procesarTecla(tecla);
}
```

2) RETO 2 – Sistema de Control de Acceso

• Análisis del problema

- Descripción general: El sistema de monitoreo con alarmas supervisa continuamente una variable física y clasifica su valor en tres niveles operativos: Normal, Advertencia y Crítico. Para cada nivel se activan indicadores visuales y/o sonoros diferenciados. Se implementa histéresis para evitar oscilaciones inestables en los umbrales de transición.

• Identificación de Entradas y Salidas

– Entradas:

- * Sensor de temperatura (señal analógica o digital).
- * Botón de silencio de alarma (opcional).

– Salidas:

- * LED verde: estado Normal.
- * LED amarillo: estado Advertencia.
- * LED rojo: estado Crítico.
- * Buzzer: alarma sonora en Advertencia y Crítico.

– Restricciones técnicas

- * Los umbrales deben ser configurables mediante constantes.
- * Implementación obligatoria de histéresis para evitar inestabilidad en los límites.
- * El sistema debe funcionar en ciclo continuo con temporización no bloqueante.
- * La lectura del sensor debe ser promediada para reducir ruido.

– Definición de umbrales y variables:

En la Tabla IV, se observa la definición de las variables con sus respectivas descripciones y justificación. Reconocer las variables a utilizar será de ayuda a la hora de realizar el código.

Table IV
DEFINICIÓN DE UMBRALES Y VARIABLES

Variable	Valor	Descripción
UMBRAL_ADVERTENCIA	const = 30°C	Umbral para activar advertencia
UMBRAL_CRITICO	const = 40°C	Umbral para nivel crítico
HISTERESIS	const = 2°C	Margen para evitar fluctuaciones
temperaturaActual	float	Valor del sensor en °C
estadoActual	enum	NORMAL, ADVERTENCIA, CRÍTICO
tiempoUltimaLectura	unsigned long	Timestamp de la lectura

– Flujograma del Sistema

El algoritmo, que se observa como Algoritmo 2, funciona en ciclo continuo, e implementa un sistema de monitoreo de temperatura basado en una máquina de estados con tres niveles: NORMAL, ADVERTENCIA y CRÍTICO. Cada 500 ms se realiza una nueva lectura del sensor, se calcula la temperatura y se determina el estado del sistema aplicando histéresis. En función del estado, se activan los indicadores correspondientes.

– Justificación técnica del microcontrolador

- * Microcontrolador Seleccionado: Arduino UNO (ATmega328P)

El mismo microcontrolador del reto anterior es suficiente para este segundo reto. Para usar el DHT21, se requiere únicamente un pin digital con protocolo one-wire.

- * Análisis de Pines Requeridos:

En la Tabla V, se observan los pines solicitados para el correcto funcionamiento del programa.

Algorithm 2 Control de Temperatura con Histéresis

```

INICIO
1: Inicializar pines, LCD, sensor
2: estadoActual ← NORMAL
3: loop PRINCIPAL (cada 500 ms)
4:   temperaturaActual ← leerSensor()
5:   if estadoActual = NORMAL then
6:     if temperaturaActual ≥ UMBRAL_ADVERTENCIA then
7:       estadoActual ← ADVERTENCIA
8:     end if
9:   else if estadoActual = ADVERTENCIA then
10:    if temperaturaActual ≥ UMBRAL_CRITICO then
11:      estadoActual ← CRITICO
12:    else if temperaturaActual < (UMBRAL_ADVERTENCIA – HISTERESIS) then
13:      estadoActual ← NORMAL
14:    end if
15:  else if estadoActual = CRITICO then
16:    if temperaturaActual < (UMBRAL_CRITICO – HISTERESIS) then
17:      estadoActual ← ADVERTENCIA
18:    end if
19:  end if
20:  actualizarIndicadores(estadoActual)
21:  mostrarEnLCD(temperaturaActual, estadoActual)
22: end loop
FIN

```

Table V
ANÁLISIS DE PINES REQUERIDOS (SISTEMA DE MONITOREO)

Componente	Pin	Tipo
Sensor NTC / DHT21	A0 o D2	Analógico / Digital
LED verde	D9	Digital OUT-PUT
LED amarillo	D10	Digital OUT-PUT
LED rojo	D11	Digital OUT-PUT
Buzzer	D8	Digital OUT-PUT
Total	6 pines	—

* Componentes auxiliares

El sistema emplea como elemento principal de medición un sensor de temperatura DHT21. La visualización de la información se realiza mediante monitor serial en el Arduino IDE, lo que optimiza

el uso de pines del microcontrolador.

Para la señalización del estado del sistema se utilizan tres LEDs de 5 mm (verde, amarillo y rojo) de 20 mA, cada uno con su respectiva resistencia limitadora de 220 ohms para protección. Además, se integra un buzzer activo de 5 V y aproximadamente 85 dB que funciona como alarma sonora en condiciones críticas, reforzando la advertencia al usuario.

* Esquema de conexión

En la Figura 4 se observa el diagrama de conexión correspondiente al funcionamiento de este circuito, desarrollado a través del programa de circuitado. A continuación se mencionan las conexiones realizadas:

- Sensor DHT21: D2 → pin DATA (resistencia pull-up 10k ohms entre DATA y VCC), VCC a 5V, GND a GND
- LED verde: D9 → 220 ohms → LED → GND
- LED amarillo: D10 → 220 ohms → LED → GND
- LED rojo: D11 → 220 ohms → LED → GND
- Buzzer: D8 → buzzer (+) → GND

EasyEDA.

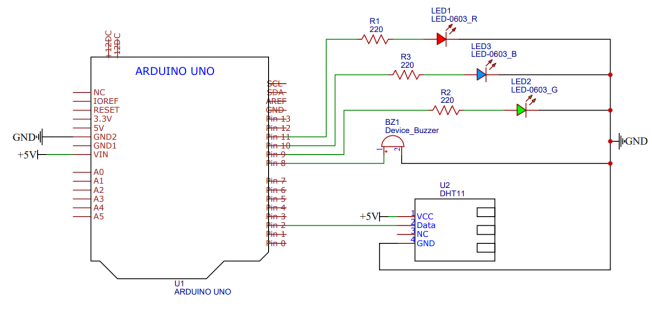


Figure 4. Esquemático de Conexión Reto 2

– Código Documentado

El código propuesto documentado se encuentra registrado a continuación:

Listing 2. Sistema de Monitoreo de Temperatura con DHT11

```

#include <DHT.h>

// --- Pines ---

```

```

const int PIN_SENSOR      = 2;
const int PIN_LED_VERDE   = 9;
const int PIN_LED_AMARILLO = 10;
const int PIN_LED_ROJO    = 11;
const int PIN_BUZZER      = 8;

// --- Configuración DHT11 ---
#define DHTTYPE DHT11
DHT dht(PIN_SENSOR, DHTTYPE);

// --- Umbrales de temperatura ---
const float UMBRAL_ADVERTENCIA = 30.0;
const float UMBRAL_CRITICO
= 40.0;
const float HISTERESIS
= 2.0;

// --- Enumeración de estados ---
enum EstadoSistema
{ NORMAL, ADVERTENCIA, CRITICO };
EstadoSistema estadoActual = NORMAL;

// --- Variables de temporización ---
unsigned long tiempoUltimaLectura = 0;
const long INTERVALO_LECTURA
= 2000;
unsigned long tiempoBuzzer
= 0;
bool estadoBuzzer
= false;

float leerTemperatura() {
    float temp = dht.readTemperature();
    if (isnan(temp)) {
        Serial.println("Error al leer
        el DHT11");
        return -999.0;
    }
    return temp;
}

void evaluarEstado(float temperatura) {
    if (temperatura == -999.0) return;
    switch (estadoActual) {
        case NORMAL:
            if (temperatura >=

```

```

                UMBRAL_ADVERTENCIA)
                estadoActual = ADVERTENCIA;
                break;
        case ADVERTENCIA:
            if (temperatura >= UMBRAL_CRITICO)
                estadoActual = CRITICO;
            else if (temperatura <
                UMBRAL_ADVERTENCIA
                - HISTERESIS)
                estadoActual = NORMAL;
            break;
        case CRITICO:
            if (temperatura < UMBRAL_CRITICO
                - HISTERESIS)
                estadoActual = ADVERTENCIA;
            break;
    }
}

void actualizarLEDs() {
    digitalWrite(PIN_LED_VERDE,
        estadoActual == NORMAL);
    digitalWrite(PIN_LED_AMARILLO,
        estadoActual ==
        ADVERTENCIA);
    digitalWrite(PIN_LED_ROJO,
        estadoActual ==
        CRITICO);
}

void gestionarBuzzer() {
    if (estadoActual == NORMAL) {
        noTone(PIN_BUZZER);
        return;
    }
    long intervalo =
        (estadoActual == ADVERTENCIA)
        ? 1000 : 250;
    if ((millis() - tiempoBuzzer)
        >= intervalo) {
        tiempoBuzzer = millis();
        estadoBuzzer = !estadoBuzzer;
        if (estadoBuzzer)
            tone(PIN_BUZZER, 1500,
                intervalo / 2);
        else

```

```

        noTone(PIN_BUZZER);
    }
}

void setup() {
    Serial.begin(9600);
    dht.begin();
    pinMode(PIN_LED_VERDE,    OUTPUT);
    pinMode(PIN_LED_AMARILLO, OUTPUT);
    pinMode(PIN_LED_ROJO,     OUTPUT);
    pinMode(PIN_BUZZER,       OUTPUT);
    Serial.println(" Sistema de
    Monitoreo - Activo");
}

void loop() {
    unsigned long ahora = millis();
    if (ahora - tiempoUltimaLectura >=
        INTERVALO_LECTURA) {
        tiempoUltimaLectura = ahora;
        float temperatura = leerTemperatura();
        evaluarEstado(temperatura);
        actualizarLEDs();
        Serial.print("T=");
        Serial.print(temperatura);
        Serial.print(" Estado=");
        Serial.println(estadoActual);
    }
    gestionarBuzzer();
}

```

3) RETO 3 – Sistema de Control de Acceso

- Análisis del problema

- Descripción general: El sistema de riego automático para cultivo de tomate monitorea continuamente la humedad relativa del ambiente mediante un sensor digital DHT11. Cuando la humedad desciende por debajo de un umbral mínimo, activa una bomba de agua (mediante relé) para iniciar el riego. El proceso se detiene al alcanzar el nivel de humedad óptimo. Se implementa histéresis para evitar ciclos de encendido/apagado rápidos y temporización no bloqueante para garantizar un funcionamiento secuencial estable. La información del sistema se visualiza en una pantalla OLED I2C de 0.96".

- Identificación de Entradas y Salidas

- Entradas:

- * Sensor DHT11: señal digital en pin D2 (humedad relativa y temperatura).
- * Botón de riego manual (override): pin digital D3.

- Salidas:

- * Módulo relé 5V: activa/desactiva la bomba de agua.
- * LED verde: humedad adecuada (IDLE).
- * LED azul/amarillo: riego activo (REGANDO).
- * LED rojo: humedad crítica baja (ALARMA).
- * Pantalla OLED I2C 0.96" 128x64: visualización del porcentaje de humedad y estado del sistema.

- Restricciones técnicas

- No se debe usar delay() en el ciclo principal.
- Implementación de histéresis en umbrales de humedad.
- El sistema debe operar de forma completamente autónoma sin intervención humana.
- Separación de la lógica de negocio del control de hardware.
- El DHT11 requiere un intervalo mínimo de 2000 ms entre lecturas.
- Tiempo mínimo de riego: 5 segundos para evitar microciclos.

- Definición de umbrales y variables:

En la Tabla VI, se observa la definición de las variables con sus respectivas descripciones y justificación. Reconocer las variables a utilizar será de ayuda a la hora de realizar el código.

- Flujograma del Sistema

El sistema opera mediante un ciclo continuo con lecturas periódicas del sensor DHT11 cada 2000 ms. La lógica de control de la bomba se ejecuta basándose en el estado actual y los valores de humedad medidos, aplicando histéresis para garantizar estabilidad.

- Justificación técnica del microcontrolador

- Microcontrolador Seleccionado: Arduino UNO (ATmega328P)

Para el sistema de riego autónomo se selecciona el

Table VI
VARIABLES INVOLUCRADAS (SISTEMA DE RIEGO AUTOMÁTICO)

Variable	Tipo	Valor Rango	/
UMBRAL_SECO	const int	30%	
UMBRAL_HUMEDO	const int	70%	
UMBRAL_ALARMA	const int	20%	
HISTERESIS	const int	5%	
humedadActual	float	0–100%	
estadoRiego	enum	IDLE / REGANDO / ALARMA	
bombaActiva	bool	true / false	
tiempoInicioRiego	unsigned long	ms	
tiempoMinRiego	const long	5000 ms	
Total	9 variables	—	

Arduino UNO. El ATmega328P es suficiente para este prototipo educativo dado que el DHT11 utiliza un único pin digital y la pantalla OLED SSD1306 se comunica por I2C, ocupando únicamente los pines A4 y A5. Para aplicaciones de campo se recomendaría el Arduino Nano o ESP32 por su menor consumo energético y posibilidad de conectividad inalámbrica.

- Análisis de Pines Requeridos:
En la Tabla VII, se observan los pines solicitados para el correcto funcionamiento del programa.
- Capacidad de Procesamiento y Consumo
 - * CPU: ATmega328P @ 16 MHz — suficiente para lectura digital y control.
 - * Consumo sistema: 85 mA (sin bomba); el DHT11 consume 1 mA y la OLED 20 mA en operación.
 - * Relé 5V: bobina 70 mA; la bomba se alimenta de fuente externa.
 - * Se recomienda fuente de 12V/2A para alimentar tanto el Arduino como la bomba.
 - * El relé desacopla galvánicamente el microcontrolador de la carga.
- Componentes auxiliares
En la tabla VIII, se observan los componentes relevantes al proyecto a desarrollar.

Algorithm 3 Algoritmo de Sistema de Riego Automático

Inicialización:

```

1: Inicializar pines, YI69, relé en OFF y pantalla OLED
2: estadoRiego = IDLE
3: bombaActiva = false
4: while true do
5:   if millis() – tiempoUltimaLectura ≥ 2000 then
6:     tiempoUltimaLectura = millis()
7:     humedadActual = dht.readHumidity()
8:     if humedadActual = invalida then
9:       Mostrar error en OLED
10:    else
11:      if estadoRiego = IDLE then
12:        if humedadActual < UMBRAL_ALARMA then
13:          estadoRiego = ALARMA
14:          activarBomba()
15:        else if humedadActual ≤ UMBRAL_SECO then
16:          estadoRiego = REGANDO
17:          activarBomba()
18:          tiempoInicioRiego = millis()
19:        end if
20:      else if estadoRiego = REGANDO then
21:        if millis() – tiempoInicioRiego ≥ tiempoMinRiego then
22:          if humedadActual ≥ UMBRAL_HUMEDO then
23:            estadoRiego = IDLE
24:            desactivarBomba()
25:          end if
26:        end if
27:      else if estadoRiego = ALARMA then
28:        activarBomba()
29:        if humedadActual ≥ (UMBRAL_SECO + HISTERESIS) then
30:          estadoRiego = IDLE
31:          desactivarBomba()
32:        end if
33:      end if
34:      actualizarIndicadores(estadoRiego)
35:      mostrarP(humedadActual, estadoRiego)
36:    end if
37:  end if
38:  gestionarBuzzerAlarma()
39:  verificarBotonManual()
40: end while

```

- Esquema de conexión En la Figura 5 se observa el diagrama de conexión correspondiente al funcionamiento de este circuito, desarrollado a través del programa de circuitado. A continuación, se mencionan los pines conectados.

Table VII
ANÁLISIS DE PINES REQUERIDOS (SISTEMA DE RIEGO AUTOMÁTICO)

Componente	Pin	Tipo	Descripción
Sensor DHT11	D2	Digital IN	Lectura de humedad relativa y temperatura
Módulo relé 5V	D4	Digital OUT	Control de bomba de agua
LED verde (IDLE)	D5	Digital OUT	Estado normal
LED azul (RE-GANDO)	D6	Digital OUT	Estado de riego activo
LED rojo (ALARMA)	D7	Digital OUT	Estado de alarma
Buzzer	D8	Digital OUT	Alarma sonora
Botón manual	D3	Digital IN	Detención manual del riego
OLED SSD1306 0.96"	A4/A5	I2C	Visualización del sistema
Total	8 pines	—	—

Table VIII
COMPONENTES AUXILIARES (SISTEMA DE RIEGO AUTOMÁTICO)

Componente	Especificación	Función
Sensor DHT11	Digital, 3.3–5V, $\pm 5\%$ HR	Medición de humedad relativa del ambiente
Módulo relé 1 canal 5V	Carga máx. 10A / 250VAC	Conmutación de la bomba de agua
Mini bomba de agua 5V	120 L/h, 0.5–1.2W	Suministro de agua al cultivo
Tubería de silicona 6mm	—	Conducción del agua
LEDs 5mm (verde, rojo, azul)	20 mA c/u	Indicadores visuales de estado
Resistencias 220 Ω	3 unidades	Limitación de corriente para LEDs
OLED SSD1306 0.96"	I2C, 128x64 px, ~ 20 mA	Visualización de datos en tiempo real
Fuente 12V / 2A	DC regulada	Alimentación del sistema completo

- * DHT11: pin DATA a D2, VCC a 5V, GND a GND
- * Relé: IN a pin D4, VCC a 5V, GND a GND; bornes COM/NO a bomba y fuente externa
- * LED verde: D5 $\rightarrow$ 220 ohms $\rightarrow$ LED $\rightarrow$ GND
- * LED azul: D6 $\rightarrow$ 220 ohms $\rightarrow$ LED $\rightarrow$ GND
- * LED rojo: D7 $\rightarrow$ 220 ohms $\rightarrow$ LED $\rightarrow$ GND
- * Buzzer activo: D8 $\rightarrow$ buzzer (+) $\rightarrow$ GND
- * Botón manual: D3 con resistencia pull-up interna $\rightarrow$ GND al presionar
- * OLED SSD1306: SDA a A4, SCL a A5, VCC a 3.3V o 5V, GND a GND

EasyEDA.

• Código Documentado

El código propuesto documentado se encuentra registrado a continuación:

Listing 3. Sistema de Riego Automatico con DHT11 y OLED

```
#include <DHT.h>
```

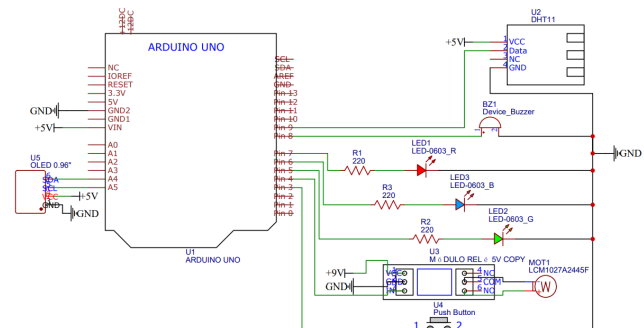


Figure 5. Esquemático de Conexión Reto 3

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
// --- Definicion de pines ---
const int PIN_SENSOR_HUMEDAD = 2;
const int PIN_RELE = 4;
const int PIN_LED_VERDE = 5;
const int PIN_LED_AZUL = 6;
const int PIN_LED_ROJO = 7;
const int PIN_BUZZER = 8;
const int PIN_BOTON_MANUAL = 3;
// --- Configuracion DHT11 ---
```

```

#define DHTTYPE DHT11
DHT dht(PIN_SENSOR_HUMEDAD, DHTTYPE);
// --- Configuracion OLED ---
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 oled(SCREEN_WIDTH,
                      SCREEN_HEIGHT,
                      &Wire,
                      OLED_RESET);

// --- Umbrales de humedad ---
const int UMBRAL_SECO = 30;
const int UMBRAL_HUMEDO = 70;
const int UMBRAL_ALARMA = 20;
const int HISTERESIS = 5;
// --- Temporizacion ---
const long INTERVALO_LECTURA = 2000;
const long TIEMPO_MIN_RIEGO = 5000;
// --- Enumeracion de estados ---
enum EstadoRiego
{ IDLE, REGANDO, ALARMA };
EstadoRiego estadoRiego = IDLE;
// --- Variables de control ---
float          humedadActual
= 0.0;
bool           bombaActiva
= false;
unsigned long tiempoUltimaLectura = 0;
unsigned long tiempoInicioRiego = 0;
unsigned long tiempoBuzzer = 0;
bool         estadoBuzzer
= false;
float leerHumedad() {
    float h = dht.readHumidity();
    if (isnan(h)) {
        Serial.println("Error DHT11");
        oled.clearDisplay();
        oled.setCursor(0, 0);
        oled.println("Error sensor");
        oled.println("DHT11 fallo");
        oled.display();
        return -1.0;
    }
    return h;
}
void activarBomba() {

```

```

    if (!bombaActiva) {
        digitalWrite(PIN_RELE, LOW);
        bombaActiva = true;
        tiempoInicioRiego = millis();
        Serial.println("BOMBA: ACTIVADA");
    }
}

void desactivarBomba() {
    if (bombaActiva) {
        digitalWrite(PIN_RELE, HIGH);
        bombaActiva = false;
        Serial.println("BOMBA: DESACTIVADA");
    }
}

void evaluarEstadoRiego() {
    bool tiempoMinCumplido =
        (millis() - tiempoInicioRiego)
        >= TIEMPO_MIN_RIEGO;
    switch (estadoRiego) {
        case IDLE:
            if (humedadActual < UMBRAL_ALARMA){
                estadoRiego = ALARMA;
                activarBomba();
            } else if (humedadActual
                       <= UMBRAL_SECO) {
                estadoRiego = REGANDO;
                activarBomba();
            }
            break;
        case REGANDO:
            if (tiempoMinCumplido &&
                humedadActual >=
                UMBRAL_HUMEDO) {
                estadoRiego = IDLE;
                desactivarBomba();
            }
            break;
        case ALARMA:
            activarBomba();
            if (humedadActual >=
                (UMBRAL_SECO + HISTERESIS)) {
                estadoRiego = IDLE;
                desactivarBomba();
            }
            break;
    }
}

```



```

}
void actualizarIndicadores() {
    digitalWrite(PIN_LED_VERDE,
                 estadoRiego == IDLE);
    digitalWrite(PIN_LED_AZUL,
                 estadoRiego == REGANDO);
    digitalWrite(PIN_LED_ROJO,
                 estadoRiego == ALARMA);
}
void gestionarBuzzerAlarma() {
    if (estadoRiego != ALARMA) {
        noTone(PIN_BUZZER);
        return;
    }
    if ((millis() - tiempoBuzzer) >= 500){
        tiempoBuzzer = millis();
        estadoBuzzer = !estadoBuzzer;
        if (estadoBuzzer)
            tone(PIN_BUZZER, 880, 250);
        else
            noTone(PIN_BUZZER);
    }
}
void verificarBotonManual() {
    if (digitalRead(PIN_BOTON_MANUAL)
        == LOW) {
        delay(50);
        if (digitalRead(PIN_BOTON_MANUAL)
            == LOW) {
            desactivarBomba();
            estadoRiego = IDLE;
            Serial.println(
                "OVERRIDE: Riego detenido");
        }
    }
}
void mostrarEnOLED() {
    oled.clearDisplay();
    oled.setTextSize(1);
    oled.setTextColor(SSD1306_WHITE);
    oled.setCursor(0, 0);
    oled.println("Sistema de Riego");
    oled.drawLine(0, 10, 127, 10,
                 SSD1306_WHITE);
    oled.setTextSize(2);
    oled.setCursor(0, 16);

    oled.print("Hum: ");
    oled.print((int)humedadActual);
    oled.println("%");
    oled.setTextSize(1);
    oled.setCursor(0, 40);
    oled.print("Estado: ");
    switch (estadoRiego) {
        case IDLE:
            oled.println("OPTIMO");    break;
        case REGANDO:
            oled.println("REGANDO");   break;
        case ALARMA:
            oled.println("ALARMA!");   break;
    }
    oled.setCursor(0, 52);
    oled.print("Bomba: ");
    oled.println(bombaActiva ?
                "ON":"OFF");
    oled.display();
}
void setup() {
    Serial.begin(9600);
    dht.begin();
    pinMode(PIN_RELE,      OUTPUT);
    pinMode(PIN_LED_VERDE, OUTPUT);
    pinMode(PIN_LED_AZUL,  OUTPUT);
    pinMode(PIN_LED_ROJO,  OUTPUT);
    pinMode(PIN_BUZZER,    OUTPUT);
    pinMode(PIN_BOTON_MANUAL,
            INPUT_PULLUP);
    digitalWrite(PIN_RELE, HIGH);
    oled.begin(SSD1306_SWITCHCAPVCC,
               0x3C);
    oled.clearDisplay();
    oled.setTextSize(1);
    oled.setTextColor(SSD1306_WHITE);
    oled.setCursor(20, 20);
    oled.println("Sistema Riego");
    oled.setCursor(25, 35);
    oled.println("Iniciando ...");
    oled.display();
    delay(2000);
    oled.clearDisplay();
    Serial.println(
        "Sistema de Riego - Listo");
}

```

```

void loop() {
    unsigned long ahora = millis();
    if (ahora - tiempoUltimaLectura
        >= INTERVALO_LECTURA) {
        tiempoUltimaLectura = ahora;
        humedadActual = leerHumedad();
        if (humedadActual < 0) return;
        evaluarEstadoRiego();
        actualizarIndicadores();
        mostrarEnOLED();
        Serial.print("Humedad: ");
        Serial.print(humedadActual);
        Serial.print("% | Estado: ");
        Serial.println(estadoRiego);
    }
    gestionarBuzzerAlarma();
    verificarBotonManual();
}

```

IV. PROTOCOLOS DE COMUNICACIÓN

- ¿Por qué no basta con "tener Wi-Fi"? Una solución IoT completa involucra múltiples capas de comunicación. Algunas se encargan de conectar sensores al microcontrolador, otras permiten enlazar el nodo con internet, y otras definen cómo se intercambian los datos a nivel de aplicación. Reducir el IoT únicamente a la conectividad Wi-Fi constituye un error conceptual frecuente, ya que ignora la complejidad real de los sistemas embebidos conectados.
- Comunicación interna del nodo embebido (dentro del hardware) Antes de transmitir información a la red, el sistema debe capturar y procesar correctamente los datos provenientes de los sensores:
 - * GPIO y señales digitales: lectura de estados y activación de actuadores.
 - * Entradas analógicas y ADC: adquisición de variables continuas como temperatura o presión.
 - * PWM: control de velocidad, intensidad luminosa o posición.
 - * I2C: comunicación con múltiples sensores usando pocos conductores.
 - * SPI: transmisión de alta velocidad con memorias, pantallas o sensores especializados.

- * UART: comunicación serial para depuración o conexión con módulos externos.
- Tecnologías inalámbricas para conectar el nodo a la red

La selección de la tecnología de comunicación depende del equilibrio entre alcance, consumo energético, velocidad de transmisión y costo de implementación. En la Tabla IV se resumen las tecnologías más utilizadas. [h]

No existe una tecnología universalmente superior; la elección depende del caso de uso específico.
- Notas específicas de cada tecnología
 - * Wi-Fi: basado en el estándar 802.11, ideal para transmisión de grandes volúmenes de datos y conexión directa a internet.
 - * Bluetooth / BLE: optimizado para bajo consumo, fundamental en dispositivos portátiles y sensores de batería.
 - * ZigBee: diseñado para redes de sensores distribuidos que operan de forma cooperativa.
 - * Thread: protocolo mallado basado en IPv6, auto-reparable y eficiente energéticamente.
 - * LoRaWAN: orientado a telemetría de largo alcance con bajo consumo y baja frecuencia de transmisión.
 - * Celular / NB-IoT: permite independencia de redes locales, aunque con mayor complejidad operativa.
- Protocolos de aplicación: cómo se empaqueta y transporta la información

Una vez establecida la conexión de red, es necesario definir el protocolo de intercambio de datos:

 - * HTTP / REST: modelo cliente-servidor basado en solicitudes y respuestas. Adecuado para servicios web y pruebas iniciales.
 - * MQTT: protocolo ligero basado en publicador/suscriptor, altamente eficiente para telemetría y sistemas distribuidos. El broker gestiona la distribución de mensajes.
 - * WebSocket: permite comunicación bidireccional continua mediante conexión persistente, útil para dashboards y control en tiempo real.

En aplicaciones con ESP32, MQTT resulta especial-

Tecnología	Alcance	Consumo	Tasa de datos	Uso típico
Wi-Fi	~100 m	Alto	Hasta 1 Gbps	Hogares, oficinas, prototipos ESP32
Bluetooth / BLE	10–100 m	Bajo	1–3 Mbps	Wearables, sensores cercanos
ZigBee	~100 m	Bajo	250 kbps	Domótica, automatización industrial
LoRa / LPWAN	2–15 km	Muy bajo	0.3–50 kbps	Agricultura, ciudades inteligentes, campo abierto
NFC / RFID	< 10 cm	Muy bajo	424 kbps	Identificación, pagos, logística
Celular 4G/5G	Km	Variable	Hasta Gbps	Vehículos, monitoreo remoto sin red local
Ethernet	Cableado	Bajo/medio	Alta	Ambientes industriales fijos, alta estabilidad

Table IX
TECNOLOGÍAS INALÁMBRICAS PARA CONECTAR EL NODO A LA RED

mente formativo para comprender la lógica IoT, mientras que HTTP facilita la integración con servicios web.

– Criterio de selección resumido

No existe una solución única óptima. La elección adecuada surge del análisis de los requerimientos del sistema:

- * ¿Qué alcance de comunicación se necesita?
- * ¿Qué volumen de datos se transmitirá y con qué frecuencia?
- * ¿Cuál es el consumo energético permitido?
- * ¿Cuál es el costo y la facilidad de implementación en el entorno específico?

V. IOT VS INTERNET CONVENCIONAL

- ¿Qué es el IoT y de dónde viene?

El término “Internet de las Cosas” fue acuñado por Kevin Ashton en 1999, en el contexto del MIT, para describir

la idea de conectar objetos físicos a internet mediante etiquetas RFID. Su consolidación como paradigma tecnológico dependió del avance simultáneo de tres factores: el desarrollo de microcontroladores cada vez más pequeños, económicos y eficientes; la masificación de la conectividad inalámbrica; y la adopción del protocolo IPv6, que permitió asignar direcciones únicas a miles de millones de dispositivos. El cambio fundamental no fue solo conectar dispositivos, sino convertir el entorno físico en una fuente permanente de datos útiles para supervisión, decisión y acción.

– Diferencia conceptual entre los tres paradigmas

En el **internet tradicional**, la interacción ocurre entre personas y servicios web: el usuario consulta, descarga o accede a plataformas. Los datos son texto, imagen, audio o video, y el flujo es persona → red → servicio. La generación de datos depende completamente de la acción deliberada del usuario humano; sin interacción, no hay dato.

En el **IoT**, los objetos físicos captan, procesan y comunican datos de forma autónoma. El entorno físico se convierte en fuente permanente de información sin requerir intervención humana directa. El flujo cambia a: sensor → procesamiento → red → decisión → acción. Los datos dominantes son variables físicas del entorno como temperatura, humedad, presión o posición. Los objetivos son el monitoreo, control y automatización distribuida. Ejemplos representativos incluyen estaciones ambientales, casas inteligentes y sistemas de riego automatizado.

En el **IIoT** (Internet Industrial de las Cosas), los nodos se integran con máquinas, procesos industriales, líneas de producción, inventarios y sistemas de trazabilidad. Los datos dominantes son variables de proceso, estado de activos, calidad y trazabilidad productiva. El objetivo central es la optimización operativa, el mantenimiento predictivo, la eficiencia energética y la producción inteligente. A diferencia del IoT doméstico o académico, el IIoT opera bajo requisitos más estrictos de confiabilidad, latencia y seguridad.

En la **Industria 4.0**, el dato deja de ser solo un registro histórico y se convierte en base para analítica avanzada, interoperabilidad entre sistemas heterogéneos, decisión en tiempo real y automatización inteligente apoyada en inteligencia artificial y gemelos digitales.

– IoT desde la perspectiva de sistemas embebidos

Un sistema embebido en el contexto IoT deja de ser un dispositivo aislado y se convierte en un nodo activo dentro de una arquitectura mayor. Para cumplir este rol, debe integrar cinco funciones fundamentales:

- * **Sensar:** captar variables físicas del entorno mediante sensores de temperatura, humedad, presión, corriente, vibración, posición, entre otros.
- * **Procesar:** acondicionar, filtrar, escalar, convertir y organizar los datos dentro del microcontrolador o en una etapa de procesamiento en el borde (*edge computing*).
- * **Comunicar:** transmitir la información hacia otros nodos o hacia internet mediante un medio físico y un protocolo de aplicación apropiados al contexto.
- * **Visualizar o almacenar:** presentar los datos al usuario final a través de dashboards o interfaces web, o registrarlos en bases de datos para análisis

posterior.

- * **Actuar:** ejecutar decisiones que modifiquen el proceso físico mediante relés, válvulas, motores, alarmas o consignas de control.

La idea clave es que el microcontrolador no es el sistema completo, sino únicamente el nodo local. El valor real del IoT aparece cuando el dato sale del dispositivo, viaja por la red, se integra con información de otros nodos y se convierte en conocimiento útil para tomar decisiones concretas.

– Arquitectura por capas de un sistema IoT

Los sistemas IoT se organizan típicamente en una arquitectura de cuatro capas, cada una con una función específica dentro del flujo del dato:

- * **Capa física o de percepción:** contiene los sensores, actuadores, el microcontrolador, conversores ADC y pines GPIO. Es el punto de contacto directo con el mundo físico. Pregunta de diseño: ¿qué se mide, con qué precisión, con qué frecuencia de muestreo y qué variables se controlan?
- * **Capa de comunicación:** se encarga de mover la información desde la capa física hacia internet o hacia otros nodos. Incluye tecnologías como Wi-Fi, BLE, Zigbee, LoRa, Ethernet, redes móviles y gateways intermediarios. Pregunta de diseño: ¿qué alcance se necesita, qué consumo energético es aceptable, qué protocolo conviene según el volumen y frecuencia de datos?
- * **Capa de servicio:** gestiona la recepción, almacenamiento, procesamiento y protección de los datos. Incluye brokers MQTT, bases de datos relacionales o de series temporales, servidores web, APIs REST, dashboards de visualización, reglas de negocio y sistemas de alertas. Pregunta de diseño: ¿dónde se almacenan los datos, cómo se visualizan en tiempo real, qué nivel de seguridad y autenticación se requiere?
- * **Capa de aplicación:** es la que entrega valor directo al usuario o al proceso. Habilita funciones de monitoreo remoto, control automatizado, predicción de fallos, mantenimiento preventivo, trazabilidad de activos y optimización logística. Pregunta de diseño: ¿qué decisión concreta permite tomar el

sistema, qué indicador operativo mejora?

– Transición IoT → IIoT → Industria 4.0 en síntesis

- * **IoT:** monitorea variables ambientales o de dispositivos individuales en contextos domésticos, urbanos o académicos. El alcance es limitado y los requisitos de confiabilidad son moderados.
- * **IIoT:** integra datos de múltiples nodos distribuidos para mejorar productividad, disponibilidad de activos, seguridad operacional y calidad del producto en entornos industriales exigentes.
- * **Industria 4.0:** el dato se convierte en el recurso central. Se habilitan capacidades de analítica avanzada, interoperabilidad entre sistemas heterogéneos, automatización inteligente y decisión en tiempo real apoyada en modelos predictivos.

– Desafíos recurrentes que no deben ignorarse

Independientemente de la escala del sistema IoT, existen desafíos transversales que deben considerarse desde las etapas iniciales del diseño:

- * **Seguridad:** proteger credenciales de acceso, cifrar los canales de comunicación y controlar el acceso a los datos almacenados. Un sistema IoT mal asegurado representa un vector de ataque crítico.
- * **Energía:** estimar el consumo de cada componente, definir la autonomía requerida y seleccionar el tipo de alimentación adecuado, especialmente en nodos remotos alimentados por batería o energía solar.
- * **Robustez:** prever y gestionar situaciones de pérdida de conexión, reinicios inesperados, errores de lectura de sensores y condiciones ambientales adversas. El sistema debe recuperarse de forma autónoma.
- * **Escalabilidad:** diseñar la arquitectura de modo que funcione correctamente con un nodo, con diez o con una red de cientos de dispositivos, sin requerir rediseños estructurales.
- * **Mantenibilidad:** documentar con claridad las conexiones físicas, las variables del sistema, los formatos de mensaje, las versiones de librerías y las dependencias de software, para facilitar el mantenimiento y la actualización futura del sistema.

VI. RESULTADOS Y DISCUSIÓN

VII. CONCLUSIONES

En esta plantilla:

- Se mostró la estructura básica de un artículo IEEEtran en LaTeX en español. - Se incluyeron ejemplos mínimos de figuras, tablas, ecuaciones y citas. - Se proporcionaron comentarios en el código para guiar el uso en clase.

Trabajo futuro: reemplazar el contenido de ejemplo por el de tu proyecto, añadir más secciones si es necesario y ajustar el preámbulo según las normas de la conferencia o revista.

AGRADECIMIENTOS

Se agradece a la institución X por el apoyo y a la agencia Y por la financiación del proyecto Z. (Sustituir por información real o eliminar esta sección si no aplica.)

REFERENCES

- [1] L. D. Verduga Monar, "Controladores con restricciones en el tiempo de respuesta implementados en un sistema embebido stm32 aplicados a las variables flujo y presión de un reactor industrial : diseño e implementación de un controlador con restricciones en el tiempo de respuesta en un sistema embebido stm32 aplicado a la variable presión de un reactor industrial simulado en matlab-simulink bajo el concepto de "hardware in the loop". Master's thesis, Escuela Politécnica Nacional, 01 2025. [Online]. Available: <https://bibdigital.epn.edu.ec/handle/15000/26639>
- [2] L. E. Wirth and F. G. Ortiz, "Desarrollo e implementación del sistema de control de bajo nivel de un robot 4wd para ambientes frutícolas," Master's thesis, Universidad Nacional del Comahue, 2022. [Online]. Available: <https://rdi.uncoma.edu.ar/handle/uncomaid/16993>
- [3] S. Salas Arriarán, *Todo sobre sistemas embebidos*. Editorial UPC, 2015. [Online]. Available: <https://editorial.upc.edu.pe/todo-sobre-sistemas-embebidos-iwlnk.html>
- [4] J. F. Ortega Pérez, "Herramienta de software embebido para la planificación de procesos masivos en tiempo real," Master's thesis, Benemérita Universidad Autónoma de Puebla, 2 2019. [Online]. Available: <https://repositorioinstitucional.buap.mx/items/60f60b46-916b-4976-8703-07f6b6f91224>
- [5] *Arquitectura de software ejecutivo en tiempo real multitarea para sistemas embebidos basada en máquinas de estados finitos*. Universidad Tecnológica de Panamá, 2012.
- [6] *APLICACIÓN DE PLANIFICACIÓN DE TIEMPO REAL HETEROGÉNEA EN SISTEMAS EMBEBIDOS, IOT Y ROBÓTICA*. Red de Universidades con Carreras en Informática, 2025. [Online]. Available: https://sedici.unlp.edu.ar/bitstream/handle/10915/184212/Documento_completo.pdf-PDFA.pdf?sequence=1&isAllowed=y
- [7] Arduino, "UNO R3 | Arduino Documentation," <https://docs.arduino.cc/hardware/uno-rev3/>, 2025, [Accessed 01-03-2026].